Embedded Data Structure
(Lecture Note 20)

Searching Algorithms (3)

2025. 5. 27.

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Sequential search : baseline algorithm**
  - **It can be applied for unsorted data**
  - **Simple and easy to understand and implement**

    - Complexity : averagely N/2 (worst N)

  - **Generalized version of sequential search is important !**

```
typedef int (*FCMP)(const void *, const void *);

int lv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lv_insert(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lv_delete(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
int lfv_search(void *key, void *base, size_t *num, size_t width, FCMP fcmp);
```

Sequential Search with LL (1/5)

KU KONKUK UNIVERSITY

▪ Search with linked list

- **Key point ➔ define a "node" structure as follows :**

```
typedef struct _node{
   struct _node *next;
}node;
```

➡ **We do not fix the type of the node value**

※ How can we insert the data in the linked list ?

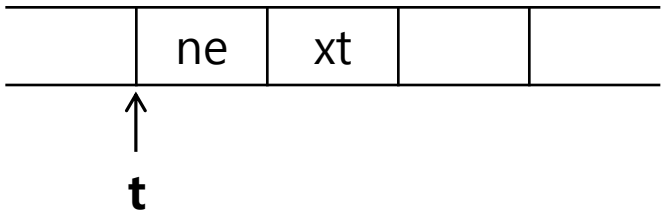
```
node *t;
t = (node *)malloc(sizeof(node) + width);
```

What does it mean ?

Where can we assign the value of our record ?

memcpy(t+1, key, width);

Where is (t+1) indicating ?



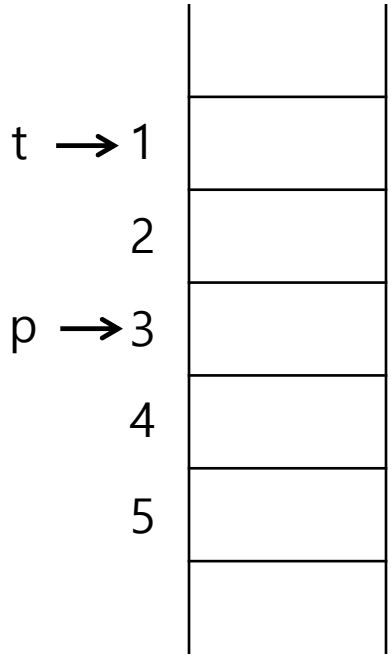
Sequential Search with LL (2/5)

- **Initialize a linked list**
  - **Based on the node pointer, we can conduct initialization**

```
void main(){
    node *t;
    init_list(    );
}
```

```
void init_list(node *p){
    p = (node *)malloc(sizeof(node));
    p->next = NULL;
}
```

```
void init_list(node **p){
    *p = (node *)malloc(sizeof(node));
    (*p)->next = NULL;
}
```



*** Draw each procedure**

*** Which one is correct ?**

Sequential Search with LL (3/5)

KU KONKUK UNIVERSITY

▪ Generalized implementation

• Sequential search for linked list

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

tyedef struct _node{ struct _node *next;} node;
typedef int (*FCMP)(const void *, const void *);

void init_list(node );
void *llv_search(void *key, node *base, size_t *num, size_t width, FCMP fcmp);
void *llv_insert(void *key, node *base, size_t *num, size_t width, FCMP fcmp);
void *llv_delete(void *key, node *base, size_t *num, size_t width, FCMP fcmp);
void *llfv_search(void *key, node *base, size_t *num, size_t width, FCMP fcmp);
```

※ Function declaration for sequential search with linked list

Sequential Search with LL (4/5)

- **Generalized implementation – cont'd**

```
void *llv_insert(void *key, node *base, size_t *num, size_t width, FCMP fcmp)
{
    node *t;
    t = (node *)malloc(sizeof(node) + width);
    memcpy(                    );
    t->next =          ;
    base->next =          ;
    (*num)++;
    return t;
}

void *llv_search(void *key, node *base, size_t *num, size_t width, FCMP fcmp)
{
    node *t;
    t =          ;
    while(fcmp(t+1, key) != 0 && t != NULL) t =          ;
    return (t == NULL ? NULL : t+1);
}
```

Sequential Search with LL (5/5)

- **Generalized implementation – cont'd**

```
#define LEN        5

Void main(){
   void *p;
   node *t, *head;
   int size = 0, key = 9;
   int data[LEN] = { 3, 1, 9, 7, 5};

   init_list(  );  // using the pointer t

   // Construct your linked list
   // Print out your linked list using for loop and printf()

   // Conduct search with key value
   // Print out the relative position of key value to the starting addr. of LL

}
```

Practical Example

- **Implementation (complete in class)**
  - **Find all the files starting with 'k' in "name.txt"**
  - **Your algorithm needs to include "insert()" and "delete()" functions**

Kathy kelvin ...

➡ **Check your result (using sequential or binary search)**

Summary

- **Sequential search in the linked list**
  - **Clearly understand the pointer operations for each function**

    ※ Why do we need to use the pointer of pointer for initialization ?
  - **Try to completely implement the generalized version**